



TXW80X Video Solution Development Guide



Notice

Due to product version upgrades or other reasons, this document will be updated from time to time. Unless otherwise agreed, this document only

As a guide for use without any warranty.

Zhuhai Taixin
Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Confidentiality level	A	TXW80X SDK Quick Start Manual	File No	TX-0000
Release date 2022-05-15			file version	V1.0

Revision history

date	Version	describe	Revised by
2022-05-15	V1.0	initial version	TX



Zhuhai Taixin Semiconductor Co., Ltd.
Zhuhai Taixin Semiconductor Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

Copyright infringement will be investigated

Copyright © 2022 by Tai Xin All rights reserved

Confidentiality level	A	TXW80X SDK Quick Start Manual	File No	TX-0000
Release date	2022-05-15		file version	V1.0

Table

of Contents

1. Overview.....	
1 2. Hardware development board.....	
1 2.1. Audio and video development board.....	2
2.1.1. Audio and video development board interface introduction.....	2
3. Video development and configuration process.....	
3 3.1. Video solution Related configuration.....	3
.....	4
.....	4
.....	4
.....	4
.....	4
4.1.1. Dvp camera.....	4
4.1.2. USB camera.....	9



Zhuhai Taixin Semiconductor
Co.,Ltd

3rd Floor, Building 11, Science and Technology Innovation Park, No. 1, Gangwan, High-tech Zone, Zhuhai City

1 Overview

This article mainly describes the video development process.

This document is mainly applicable to the following engineers:

- Technical support engineer
- Solution software development engineer

Product scope applicable to this document:

model	encapsulation	Package
TXW806	QFN56/48/40	

2. Hardware development board

For quick start and solution evaluation, we provide development boards for various application scenarios.

2.1. Audio and video development board

2.1.1. Audio and video development board interface introduction

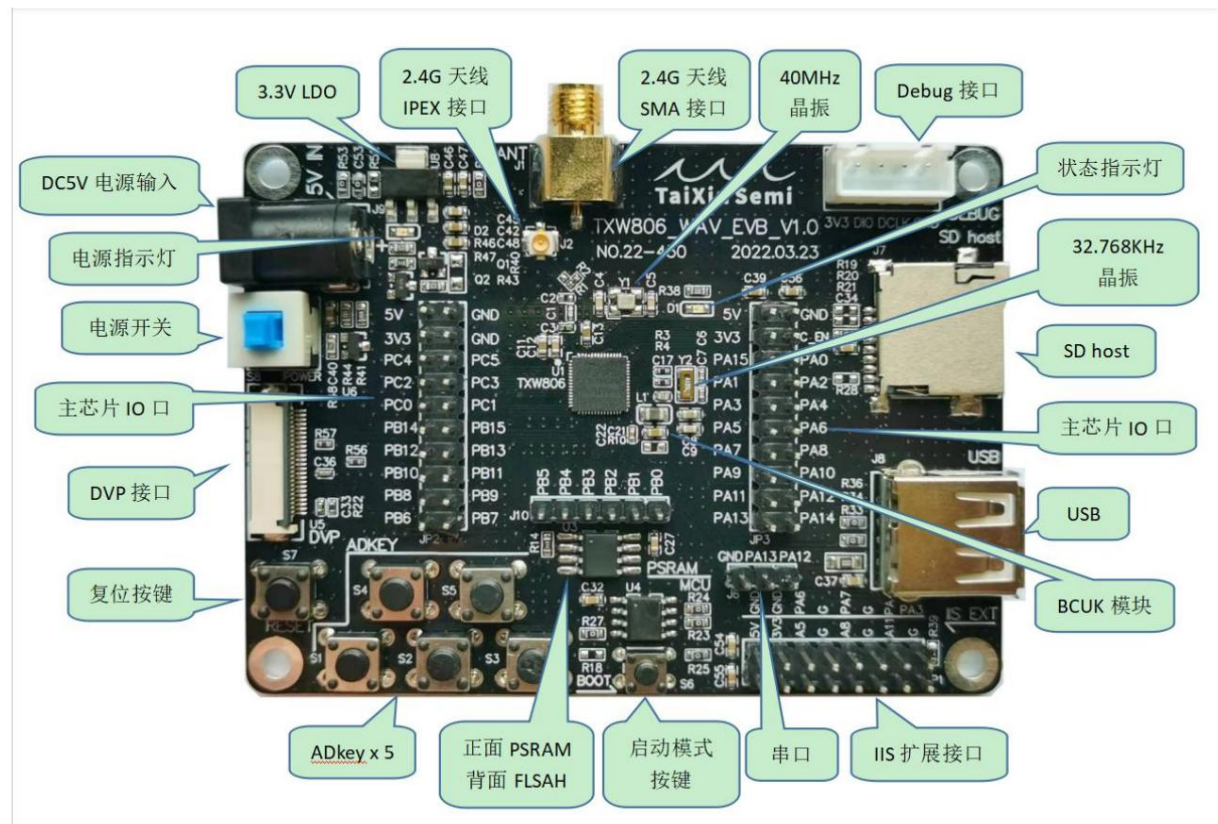


Figure 3.1.1.1 - Main view of audio and video development board

Special Instructions

Mode start button: This button can save the system with one click, run to death when the chip is powered on, cklinc programming and other upgrades

Used when all levels are disabled.

IIS extension interface: This interface is used to expand the audio daughter board.

3. Video development and configuration process

3.1. Video solution related configurations

The solution configuration mainly depends on the project_config.h configuration. The following configurations are all based on the SDK-based image transmission protocol.

```
#define USB_HOST_EN          1
#define MACBUS_USB

#define USB_EN              0
#define DVP_EN              0
#define JPG_EN              (1 && DVP_EN)
#define SDH_EN              1
#define FS_EN               1
#define VCAM_EN             (0 || DVP_EN)

#define FRAME_MANAGEMENT    1
#define OPENDML_EN          1
#define UART_INPUT_EN       1
#define UART_FLY_CTRL_EN    0
#define PWM_EN              0

#define AUDIO_EN             (0 && FRAME_MANAGEMENT)
#define IIS_AUDIO_EN        (0 && FRAME_MANAGEMENT)
#define RTP_SOUND           (1 && (AUDIO_EN || IIS_AUDIO_EN))

|
#define MJPEG_VIDEO          (1 && FRAME_MANAGEMENT && OPENDML_EN && FS_EN && SDH_EN && JPG_EN) //基于框架的mpeg录像
#define UVC_VIDEO           (1 && FRAME_MANAGEMENT && OPENDML_EN && FS_EN && SDH_EN && USB_EN) //基于框架的uvc录像
```

1. USB_EN starts the USB camera (usb and dvp cannot coexist temporarily)
2. DVP_EN starts the dvp camera (usb and dvp cannot coexist temporarily)
3. SDH_EN and FS_EN are sd card and file system initialization
4. The FRAME_MANAGEMENT (only supports SDK's image transmission protocol) framework is started. When plugging in psram, it needs to be enabled.
5. OPENDML_EN needs to be activated when recording is required (only supports the recording format of SDK, other formats need to be

accomplish)

4.1.1. DVP camera

4.1.1.1. Dvp camera related parameter configuration

1. When using a VGA camera, IMAGE_W and IMAGE_H are configured as VGA. Corresponding macro definition #define

CMOS_AUTO_LOAD 1 Open (located in project_config.h), you can automatically select all supported cameras, if

If you do not use automatic selection and only want to support a certain camera, open the macro of the corresponding camera, for example: #define

DEV_SENSOR_GC0308 1 (located in project_config.h), supported cameras and macros are defined in csi.h, as shown in the figure:

```

#ifndef DEV_SENSOR_OV7725
#define DEV_SENSOR_OV7725 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_OV7670
#define DEV_SENSOR_OV7670 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_GC0308
#define DEV_SENSOR_GC0308 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_GC0309
#define DEV_SENSOR_GC0309 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_GC0328
#define DEV_SENSOR_GC0328 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_BF3A03
#define DEV_SENSOR_BF3A03 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_BF3703
#define DEV_SENSOR_BF3703 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_OV2640
#define DEV_SENSOR_OV2640 0
#endif

#ifndef DEV_SENSOR_BF2013
#define DEV_SENSOR_BF2013 (0 | CMOS_AUTO_LOAD)
#endif

#ifndef DEV_SENSOR_XC7016_H63
#define DEV_SENSOR_XC7016_H63 0
#endif

#ifndef DEV_SENSOR_XC7011_H63
#define DEV_SENSOR_XC7011_H63 0
#endif

#ifndef DEV_SENSOR_XC7011_GC1054
#define DEV_SENSOR_XC7011_GC1054 0
#endif

#ifndef DEV_SENSOR_XCG532
#define DEV_SENSOR_XCG532 0
#endif

#ifndef DEV_SENSOR_GC2145
#define DEV_SENSOR_GC2145 0
#endif

```

2. There are two types of 720p camera processing.

- 1) External isp chip
- 2) Comes with ISP function camera

If the camera you need to support is a camera without ISP function, you need to connect an external ISP chip. Currently, it is suitable for two models.

isp chips, XC7016 and XC7011. Of course, if the chip directly supports the isp function, it can normally output the processed

YUV data can be directly connected to this main control. In the configuration, IMAGE_W and IMAGE_H are configured as 720P.

4.1.1.2. Dvp camera added

Please refer to the existing camera configuration file sensor_XXX.c, add the camera configuration file first, and record

Pair of data structures _Sensor_Adpt_ and _Sensor_Ident

_Sensor_Ident contains the read and write commands of the current camera, command and data operation length, ID address and ID

value, use this configuration information in sensorCheckId to determine whether the currently inserted camera matches this configuration.

_Sensor_Adpt_ contains important information including pixelw, pixelh, hsyn, vsyn, init, mclk, pixelw

The input resolution configured with pixelh for the current image. hsyn and vsyn are the current camera configurations, hsync

The effective level of vsync is high level or low level. mclk is the current configuration. When the camera is inserted, the system gives

mclk frequency provided by this camera. init is the initialization configuration information of the current camera. The default configuration output is

~~~~~

After configuring the camera configuration file sensor\_XXX.c, please add the corresponding statement to csi.h.

And add corresponding camera information to devSensorInitTable and devSensorOPTTable

#### 4.1.1.3. Dvp camera mjpeg encoding parameter configuration

1. The total size of mjpeg is configured as JPG\_NODE\*(JPG\_BUF\_LEN+JPG\_TAIL\_RESERVER), macro definition reference

project\_config.h file, JPG\_BUF\_LEN cannot be too small, this determines the interrupt frequency of mjpeg

rate, the larger JPG\_BUF\_LEN (configured according to the remaining memory space), the fewer interruptions of a complete picture, and the efficiency is

The fewer errors there are, JPG\_NODE is the number of nodes, and the mjpeg interrupt will obtain nodes from the node pool of JPG\_NODE, such as

If there are not enough nodes, the currently encoded picture will be lost due to insufficient memory. JPG\_TAIL\_RESERVER is used for

Custom parameters, if not needed, set to 0. JPG\_HEAD\_RESERVER is reserved for the header and is also customized, no need

If it can be set to 0, it is the default 24 in the official sdk, which will be used in the image transmission protocol of the sdk itself.

```

:
: #if FRAME_MANAGEMENT
: uint8 psram_jpg_buf[JPG_NODE*(JPG_BUF_LEN+JPG_TAIL_RESERVER)]__attribute__((section(".psram.src")));
: #else
: uint8 psram_jpg_buf[JPG_NODE*(JPG_BUF_LEN+JPG_TAIL_RESERVER)];__attribute__((section(".psram.src")));
: #endif
:

```

2. For image compression, please refer to the function jpg\_quality\_tidy. This function is called in an interrupt and is used to pass

The size of the current picture is used to control the adjustment direction of the image quality. It can be understood that this function is the control of jpg picture size.

control scope. However, this function is only an expected value and may not necessarily be able to control a range that is actually too large. For example, if 720P

The compressed size of the image is controlled within 20K. This kind of image cannot be compressed to this size. The compressed size is the same as

The currently used quantization table is related to the fine-tuning value. For the quantization table, please refer to quality\_tab. Currently, 6 quantization tables are provided.

The table can be modified by customers if necessary. Fine-tuning means that in addition to this quantitative table, it can also be based on the current usage



The quantization table is used to fine-tune the data at 16 levels. Level 8 means the quantification table remains unchanged, 1~7 means upward optimization of quality, and 9~F means quality.

Volume adjustment downward.

#### 4.1.1.4. Dvp image acquisition process

The dvp image acquisition interface is as shown in the figure: (jpgdef.h)

```

0:  #ifndef JPG_LEN == 1
7:  extern int get_jpg_node_len();
8:  #define GET_NODE_LEN(f) get_jpg_node_len(f)
9:  #define GET_JPG_LEN(f) get_jpeg_len(f)
0:  #define GET_JPG_buf(f) get_jpeg_first_buf(f)
1:  #define GET_JPG_FRAME() get_frame()
2:  #define FREE_JPG_NODE(f) del_jpeg_first_node(f)
3:  #define DEL_JPG_FRAME(f) del_jpeg_frame(f)
4:

```

1. First get the picture frame `f = GET_JPG_FRAME()`. If it is obtained, it means that the entire picture has been obtained.

The composition is connected in the form of nodes, and the entire graph can be obtained by reading in a loop.

2. `GET_JPG_LEN(f)` gets the size of a picture
3. `GET_JPG_buf(f)` gets the first node of the picture frame,
4. `FREE_JPG_NODE(f)` releases the first node of the picture frame
5. `GET_NODE_LEN(f)` gets the length of a node (the length of all nodes in the picture frame is the same, so when the last node of the picture frame is obtained, the actual content  $\leq$  `GET_NODE_LEN(f)`, so when processing the last frame, the actual content size of the last node should be calculated based on the image size)
6. `DEL_JPG_FRAME(f)` deletes the picture frame and returns all nodes at the same time. After the picture frame is used, you need to call this interface to release it, otherwise the picture frame will be lost and the picture frame will not be obtained subsequently.

pseudocode:

```

int read_size = 0;
int photo_size_tmp, photo_size, node_len;
uint8 *buf;
//获取图片帧
void *f = GET_JPG_FRAME()
if(f)
{
    //获取节点长度
    node_len = GET_NODE_LEN(f);
    //获取图片的size
    photo_size_tmp = photo_size = GET_JPG_LEN(f);

    //循环读取图片帧的节点
    while(photo_size_tmp)
    {
        buf = GET_JPG_buf(f);
        //异常,不能出现这种情况
        if(!buf)
        {
            printf("get buf err\n");
            while(1);
        }
        if(photo_size_tmp >= node_len)
        {
            read_size = node_len;
        }
        else
        {
            read_size = photo_size_tmp;
        }
        //保存图片的buf,自定义实现,buf read_size

        FREE_JPG_NODE(f);
        photo_size_tmp -= read_size;
    }

    //图片帧使用完毕
    DEL_JPG_FRAME(f)
}

```

#### 4.1.1.5. Dvp data flow description

The generation steps and basic ideas of DvpJPG data stream are as

follows: 1: JPG data buf is allocated in the form of nodes.

2: DVP collects enough row data

3: JPG extraction and

compression 4: When the amount of compressed JPG data reaches the configured data buf length or the image acquisition is completed, an out\_buf\_full or done interrupt is generated (for details, please see the jpg module api function description) 5:

Record the node status in the interrupt and Allocate the node position 6 where the next jpg data will be stored: If

the done interrupt is completed, the application can extract the corresponding linked list data and reorganize the jpg image according to the instructions in 4.1.1.3

Among them, jpg data buf node allocation, please check the jpg\_room\_init function. Here is a special explanation. There are two frame root nodes, namely jpg\_frame[2], which are pointed to by jpg\_p (interrupt) and usr\_p (application) pointers respectively, used to indicate the use of the corresponding frame. Is it interrupting or applying. In addition to the two root nodes of jpg\_frame, there is also a root node of free\_tab, which is used to manage many spatial sub-nodes generated by free\_table\_init. Each space sub-node will be bound to its own exclusive space, and the image data is stored in these many spaces. The entire frame of image data is not continuous, but the image data arrangement can be found and reorganized according to the application node of jpg\_frame.

#### 4.1.1.6. Dvp important printing instructions

1. Print information "sip reset DVP", DVP acquisition error, please check whether the camera is plugged in, or the configuration table

Check whether the information in IMAGE\_W and IMAGE\_H is consistent. If IMAGE\_W and IMAGE\_H are configured as 720P and the configuration table is configured as VGA, this print will occur because there is a problem with the collected data. There is another situation where the effective levels of hsync and vsync in the configuration table are incorrectly configured, that is, the effective levels of hsync and vsync in the configuration table do not match the configuration in \_Sensor\_Adpt\_, which will also cause sampling errors.

2. Print information "fh" and "fv", the DVP processing speed cannot keep up with the collection speed, "fh" means that half of the last collected buf has been rewritten before the data is processed. "fv" means that the last collected buf has been overwritten before the data is processed. This problem may occur because the CLK allocation of DVP is too high and the CPU cannot handle it, or there are too many things processed in the DVP interrupt, which affects the processing efficiency.
3. Print "jpg done len err". There are two possibilities for this situation. One is that the CPU cannot handle it, that is, the compression rate of JPG is not high, the image is too large, and the CPU only responds once after multiple interruptions. The second situation is that the software buf space is insufficient and the nodes are not enough (refer to the data flow description). For the first case, the processing method is to increase the running speed of the code (for the running speed of xip, please check the "TXW80X TXProgrammer Tool Usage Document"), increase the space size of the node, or reduce the quality of the image. In the second case, the solution is to increase the code running speed, reduce the image quality, or increase the number of nodes if space permits.
4. Print "?". In this case, the JPG is abnormal. The code will reset DVP and JPG. If the JPG output keeps printing this problem, please check whether the FRAME\_MANAGEMENT is set to 1 and the programming code is not correct. correct. When FRAME\_MANAGEMENT is set to 1, please burn project\_merge.bin, otherwise psram will not be initialized, and jpeg data transmission to psram will be invalid, resulting in jpg exception.

## 4.1.2. USB camera

### 4.1.2.1. USB camera parameter configuration

1. Macro definition: UVC\_BLANK\_LEN is the size of the node, UVC\_BLANK\_NUM is the number of nodes, in the absence of psram,

UVC\_BLANK\_NUM needs to be large enough, UVC\_BLANK\_NUM\*UVC\_BLANK\_LEN needs at least the size of 1.5 pictures

size, otherwise the picture will be discarded. In the case of psram, UVC\_BLANK\_NUM can be smaller, and finally all pictures

The data will be placed in psram. UVC\_BLANK\_LEN should not be too small. The default is 12\*1024 byte. UVC\_FRAME\_NUM represents

The frame of the picture, the default value is 2

```

1:
2: #if USB_EN
3: extern uint8_t uvc_dat[1024];
4: #define UVC_BLANK_LEN      12*1024
5: #if UVC_PSRAM == 1
6: #define UVC_BLANK_NUM      3
7: #else
8: #define UVC_BLANK_NUM      7
9: #endif
10: #define UVC_FRAME_NUM     2
11: UVC_BLANK uvc_blank[UVC_BLANK_NUM];
12: uint8 blank_space[UVC_BLANK_NUM][UVC_BLANK_LEN+UVC_HEAD_RESERVER]__attribute__((aligned(4)));

```

In terms of resolution selection, please modify the return value in uvc\_default\_dpi for the default resolution. There are currently two default resolutions.

UVC\_VGA and UVC\_720P. If you want to modify the camera resolution during transmission, please use the USB ioctl function, for example

like:

```

18 void uvc_ioctl_index(uint8 uvc_idx){
19     uint8 msgbuf[1];
20     uint8* pt;
21
22     struct usb_device *dev;
23     dev = (struct usb_device *)dev_get(HG_USBDEV_DEVID);
24     pt = msgbuf;
25     pt[0] = uvc_idx;
26     usb_host_uvc_ioctl(dev,USB_HOST_IO_CMD_SET_IDX,msgbuf,0);
27 }

```

If the camera is connected to the hub, IOCTL configuration can also be used, USB\_HOST\_IO\_CMD\_INSERT\_HUB function.

#### 4.1.2.2. USB camera acquisition process

(The actual macro definition is the same as that of the DVP camera, and the calling function name is changed to the implementation of

UVC) The UVC image acquisition interface is as shown in the figure: (jpgdef.h)

```

extern int get_node_uvc_len();
#define UVC_HEAD_RESERVER 24 //每段JPEG BUF前面预留用于填充其他数据的数据量，无需预留填0
#define UVC_PSRAM 0
#define GET_NODE_LEN(f) get_node_uvc_len(f)
#define GET_JPG_LEN(f) get_uvc_frame_len(f)
#define GET_JPG_buf(f) get_uvc_buf(f)
#define GET_JPG_FRAME() get_uvc_message_gloal()
#define FREE_JPG_NODE(f) free_uvc_blank_nopsram(f)
#define DEL_JPG_FRAME(f) uvc_sema_up()

```

1. First obtain the picture frame `f = GET_JPG_FRAME()`. If obtained, it means that the entire picture has been obtained. The composition of the picture is connected in the form of nodes. The entire picture can be obtained by reading in a loop.
2. `GET_JPG_LEN(f)` gets the size of a picture
3. `GET_JPG_buf(f)` gets the first node of the picture frame,
4. `FREE_JPG_NODE(f)` releases the first node of the picture frame
5. `GET_NODE_LEN(f)` gets the length of a node (the length of all nodes in the picture frame is the same, so when the last node of the picture frame is obtained, the actual content  $\leq$  `GET_NODE_LEN(f)`, so when processing the last frame, the actual content size of the last node should be calculated based on the image size)
6. `DEL_JPG_FRAME(f)` deletes the picture frame and returns all nodes at the same time. After the picture frame is used, you need to call this interface to release it, otherwise the picture frame will be lost and the picture frame will not be obtained subsequently.

pseudocode:

```

int read_size = 0;
int photo_size_tmp, photo_size, node_len;
uint8 *buf;
//获取图片帧
void *f = GET_JPG_FRAME()
if(f)
{
    //获取节点长度
    node_len = GET_NODE_LEN(f);
    //获取图片的size
    photo_size_tmp = photo_size = GET_JPG_LEN(f);

    //循环读取图片帧的节点
    while(photo_size_tmp)
    {
        buf = GET_JPG_buf(f);
        //异常,不能出现这种情况
        if(!buf)
        {
            printf("get buf err\n");
            while(!);
        }
        if(photo_size_tmp >= node_len)
        {
            read_size = node_len;
        }
        else
        {
            read_size = photo_size_tmp;
        }
        //保存图片的buf,自定义实现,buf read_size

        FREE_JPG_NODE(f);
        photo_size_tmp -= read_size;
    }

    //图片帧使用完毕
    DEL_JPG_FRAME(f)
}

```

#### 4.1.2.3. USB Important Printing Instructions

1. Print "babble error". This print indicates that the signal line is in poor contact. The system will reset the usb. If the babble error continues to be printed, you need to determine whether the USB cable is normal.
2. Print "disconnect.....". This print is when the device is disconnected. This kind of disconnection is caused by poor contact or active power strip disconnection, and the system will restart. reset usb. 3. Print "XXX Err" and "Enum ERR". Such printing means that the device failed to enumerate during the enumeration process, and the system will reset the USB again. 4. Print "uvc can not set resolution:720p". This type of printing is for cameras that do not support 720P. If the 720P output is configured on the head, it will be changed to VGA output by default.
5. Print "\_D3\_". This type of printing is an error in the UVC image of the camera. If this part is printed too much, please make sure the camera is normal. 6. When printing "\_D1\_" and "\_D2\_", it indicates that the current data cache space is insufficient. If you are running a solution without psram and continue to print this, please confirm whether the image size exceeds the default size, 7\*12K. If Yes, please modify the output size of the camera. If this print does not appear continuously but also occurs frequently, it indicates that the CPU cannot process it. Please improve the running efficiency of the code (for the running speed of XIP, please check the "TXW80X TXProgrammer Tool Usage Document")